

第4回 Web勉強会

コインクリッカーで学ぶ保存とアンロック

今日のゴール

- ブラウザで動くゲームを完成させる
- クリックでコインが増える仕組みを実装する
- `localStorage` でデータを保存・復元できるようにする
- 条件に応じてUIを切り替えるアンロック機能を追加する

本日の流れ

1. 前提と環境の確認
2. HTMLテンプレートの読み解き
3. JavaScriptの基本処理を実装
4. `localStorage` で保存・復元
5. アンロック機能の追加
6. 発展課題とテストプレイ

はじめに: 環境の確認

- Webブラウザ（Chrome / Edge / Firefox）
- VS Codeなどのエディタ
- 第4回資料フォルダが準備されている
- `10_テンプレート.html` が開けること

HTMLテンプレートの構成

- `input#playerName` : プレイヤー名入力欄
- `button#clickButton` : コインを増やすボタン
- `span#coinCount` : 現在のコイン数表示
- `button#saveButton` : データ保存ボタン
- `div#unlockArea` : 条件で表示される隠し領域

HTMLのポイント

- `id` が JavaScript からの接続キーになる
- `style.display` で要素を隠したり表示したりできる
- `script` の読み込み順に注意する

JavaScriptの基本: クリックで増やす

```
let playerName = '';  
let clickCount = 0;  
  
function plusCoin() {  
  clickCount += 1;  
  updateDisplay();  
  checkUnlock();  
}
```

- `clickCount` でコイン数を管理
- `plusCoin()` に処理をまとめると分かりやすい

表示を更新する関数

```
function updateDisplay() {  
    document.getElementById('coinCount').textContent = clickCount;  
}
```

- 画面表示と内部データを分ける
- 値を変更したら必ず `updateDisplay()` を呼ぶ

クリックとイベントの接続

```
document.getElementById('clickButton').addEventListener('click', function() {  
    plusCoin();  
});
```

- ボタンのクリックを「聞く」
- クリック時に `plusCoin()` を呼び出す

localStorage で保存する

- ブラウザのローカル領域にデータを保存
- リロードしても値が残る

```
function saveGame() {  
  const data = {  
    playerName: document.getElementById('playerName').value,  
    clickCount: clickCount,  
  };  
  localStorage.setItem('myGameData', JSON.stringify(data));  
}
```

localStorage から読み込む

```
function loadGame() {  
  const saved = localStorage.getItem('myGameData');  
  if (saved) {  
    const data = JSON.parse(saved);  
    document.getElementById('playerName').value = data.playerName || '';  
    playerName = data.playerName || '';  
    clickCount = data.clickCount || 0;  
    updateDisplay();  
    checkUnlock();  
  }  
}
```

- `JSON.stringify` / `JSON.parse` が重要
- 保存されていない場合の `null` チェックを忘れない

アンロック機能の基本

- 100枚到達で隠し領域を表示
- UIを条件で切り替えるゲーム性

```
function checkUnlock() {  
  const unlockArea = document.getElementById('unlockArea');  
  if (clickCount >= 100) {  
    unlockArea.style.display = 'block';  
  } else {  
    unlockArea.style.display = 'none';  
  }  
}
```

アンロック後の工夫例

- メッセージを変える
- `disabled` を切り替える
- 新しいボタンやアイテムを表示する
- さらに `clickCount` による条件分岐を増やす

進め方と演習の流れ

1. `10_テンプレート.html` を開く
2. 画面の HTML を確認する
3. `11_配布用_虫食い.js` に実装を追加する
4. `saveGame()` / `loadGame()` を完成させる
5. `checkUnlock()` を実装して表示を切り替える
6. 実装後にリロードして保存が機能するか確認

発展課題

- autoCPS で自動増加を追加
- アイテム購入機能を追加
- クリックアニメーションや見た目を改良
- テーマを変えて自分だけのゲームにする

テストプレイと振り返り

- 自分のゲームを実際に遊んでみる
- 友達の工夫を見てアイデアを交換する
- どこが楽しかったか、どこを改善したいかを考える

よく使うキーワード

- DOM 操作
- イベントハンドリング
- 関数化
- 状態管理
- 条件分岐
- 永続化
- テンプレート化

次回へのつながり

- 今回の `save/load` を土台に、次回は
- 自分のデータをサーバーに送る (POST)
- みんなのランキングを表示する (GET)

まとめ

- クリックでコインを増やせるようになった
- `localStorage` でページをまたいてもデータが残る
- アンロック機能で画面を動的に変えられる
- これらがゲーム制作の土台になる